ClusterKV: Manipulating LLM KV Cache in Semantic Space for Recallable Compression

Guangda Liu, Chengwei Li, Jieru Zhao†, Chenqi Zhang, Minyi Guo
School of Computer Science, Shanghai Jiao Tong University
† Corresponding author: zhao-jieru@sjtu.edu.cn

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11132479

***Abstract*—Large Language Models (LLMs) have been widely deployed in a variety of applications, and the context length is rapidly increasing to handle tasks such as long-document QA and complex logical reasoning. However, long context poses significant challenges for inference efficiency, including high memory costs of key-value (KV) cache and increased latency due to extensive memory accesses. Recent works have proposed compressing KV cache to approximate computation, but these methods either evict tokens permanently, never recalling them for later inference, or recall previous tokens at the granularity of pages divided by textual positions. Both approaches degrade the model accuracy and output quality. To achieve efficient and accurate recallable KV cache compression, we introduce ClusterKV, which recalls tokens at the granularity of semantic clusters. We design and implement efficient algorithms and systems for clustering, selection, indexing and caching. Experiment results show that ClusterKV attains negligible accuracy loss across various tasks with 32k context lengths, using only a 1k to 2k KV cache budget, and achieves up to a 2× speedup in latency and a 2.5× improvement in decoding throughput. Compared to SoTA recallable KV compression methods, ClusterKV demonstrates higher model accuracy and output quality, while maintaining or exceeding inference efficiency. Our code is available at https://github.com/sjtu-zhao-lab/ClusterKV.**

I. Introduction

Large Language Models (LLMs) have been widely deployed in a variety of applications, such as natural language understanding, coding copilots and chatbots. As LLMs are used for more complex tasks, the need for long context length is rapidly increasing to handle tasks such as long-document QA, repository-level code understanding, and complex logical reasoning [1–3]. This drives LLMs to support larger context windows, expanding from the original 4k to 32k [4, 5], 128k [6] and even up to 1M [7]. While models can support larger context window with continual training or extrapolation [5], inference with long contexts poses significant challenges on efficiency. As the size of key-value (KV) cache increases linearly with the context length, it can exceed the capacity of GPU memory, leading to inference failure or extremely high latency. Since the autoregressive decoding stage is typically the performance bottleneck of LLM inference and is memory-bound [8], accessing the KV cache of all previous tokens results in increased latency for long-context inference.

To mitigate the issues, recent works compress KV cache by selecting a subset of tokens and utilizing their keys/values to approximate attention computation. The tokens are typically selected with fixed patterns [9] or based on their attention weights [10–14], which represent each token's importance in attention computation. In most compression methods, unimportant tokens are evicted permanently; once evicted, they are never recalled in subsequent decoding steps, as shown in Fig. 1b. However, the token importance is dynamically changing during decoding: tokens initially considered unimportant and evicted may become important at later steps and vice versa. Non-recallable KV cache compression greedily evicts tokens based on positions or attention weights of tokens at the current decoding step, failing to capture the dynamic feature. This can degrade model accuracy and output quality.

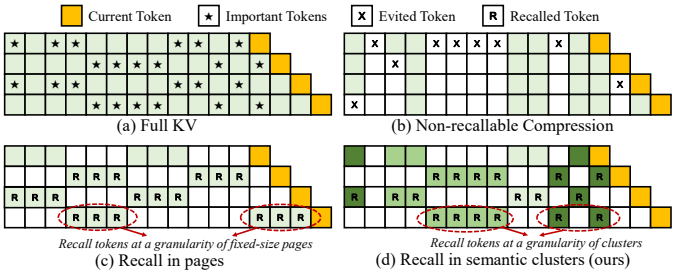

Fig. 1: Comparison of KV compression methods. Green boxes represent tokens selected for attention computation.

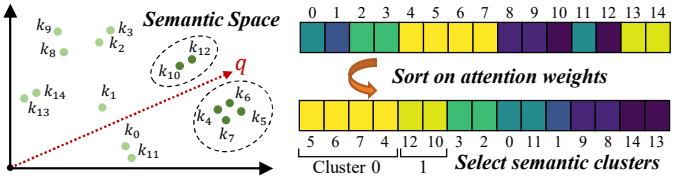

Fig. 2: Semantic space and attention weights of tokens in the last step of Fig. 1d. Lighter boxes indicate larger weights.

While making tokens recallable is promising, it incurs prohibitively high cost due to the computation of attention weights over all previous tokens. To reduce the cost, as illustrated in Fig. 1c, Quest proposed recalling at the granularity of pages [15]. A page consists of $page\_size$ consecutive tokens, allowing Quest to reduce the selection overhead by a factor of $1/page\_size$. However, since pages are divided simply by textual positions of tokens, internal fragmentation becomes an issue: a recalled page may contain unimportant tokens, wasting budget that could be allocated to truly important tokens.

To achieve efficient and accurate KV cache compression, we introduce ClusterKV, which recalls tokens at the granularity of token clusters in the semantic space, referred to as *semantic clusters*. As illustrated in Fig. 2, tokens that are close in their semantic space or key vector space exhibit similar attention weights. Therefore, as shown in Fig. 1d, ClusterKV

recalls tokens at the granularity of semantic clusters to ensure more precise token recall. Moreover, ClusterKV incorporates specialized system designs and optimized kernels to minimize recall overheads. Experimental results demonstrate that ClusterKV attains negligible accuracy loss across various tasks with context lengths up to 32k, using only a 1k to 2k KV cache budget. It also delivers up to a 2× speedup in latency and a 2.5× improvement in decoding throughput. Compared to state-of-the-art recallable KV compression methods, ClusterKV achieves significantly higher model accuracy and output quality, while maintaining or surpassing inference efficiency.

II. Background and Motivation

A. LLM Inference and KV Cache

LLMs encompass multiple Transformer layers, each containing a multi-head attention (MHA) module and a feed-forward network (FFN) with residual connections and normalization operations [16]. In MHA, the input tensor is linearly projected into query, key and value tensors ($Q, K, V \in \mathbb{R}^{N\times d}$) for each head, where N is the input length and d represents the number of channels or hidden dimensions per head. The MHA output is defined as $softmax(\frac{QK^T}{\sqrt{d}})V$, with outputs of all heads concatenated for subsequent FFN and normalization.

For generative inference, LLMs produce tokens in an autoregressive manner, appending each generated token to the input to generate the next token. To avoid the recomputation of K and V of previous tokens, these tensors are stored in memory for reuse, which is known as *KV cache*. The LLM inference includes two stages: prefill and decoding. The prefill stage processes the entire input sequence, computes KV cache and generates the first output token. During decoding, the query vector q of the latest generated token and K, V of previous tokens are used to compute attention for generating the next token, formulated as $softmax(\frac{qK^T}{\sqrt{d}})V$, where $q \in \mathbb{R}^{1\times d}$, $K, V \in \mathbb{R}^{L\times d}$ and L is the context length of previous tokens.

B. Long Context Inference and KV Cache Compression

Long context is an emerging trend of LLM inference, such as long-document QA [1, 17], repository-level code understanding [2] and complex logical reasoning [3]. Moreover, supported context windows of LLMs are extending rapidly, from 4k tokens of GPT3.5 and Llama-2 to 32k [5], 128k [6] and even 1M tokens [7]. However, long-context inference incurs significant memory and computation cost. The size of KV cache and complexity of attention computation during decoding increase linearly with context length, resulting in low inference efficiency or even inference failures.

Recent works reveal the sparsity of attention computation, i.e., only a small subset of tokens contributes to most of attention outputs [10, 13]. This observation enables KV cache compression by selecting a subset of tokens to approximate attention computation, which is formulated as $softmax(\frac{qK_S^T}{\sqrt{d}})V_S$, where $K_S, V_S \in \mathbb{R}^{B\times d}$ represents the keys and values of selected tokens, and B is the budget size of KV cache after compression. By setting a fixed budget, the KV cache size and decoding cost remain stable, regardless of the context length.

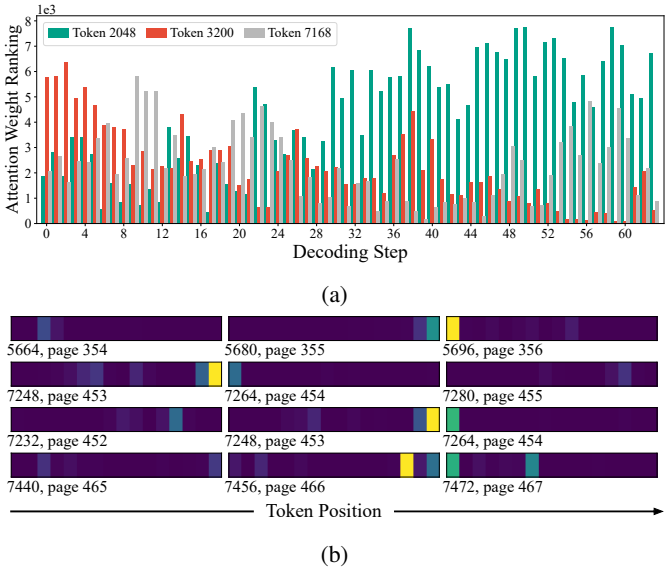

Fig. 3: (a) Variation in token importance across decoding steps with a context length of 8192. (b) Internal fragmentation of important tokens at the granularity of pages ($page\_size = 16$).

Typically, K_S and V_S are selected based on attention weights (qK^T), as tokens with higher attention weights contributes more to the attention computation [10–12].

C. Related Work and Motivation

KV cache compression should be recallable. While selecting a subset of KV cache for computation ensures inference efficiency, computing all the attention weights for selection introduces substantial overhead. Therefore, most existing works only compute attention weights over tokens that have been selected, rather than for all previous tokens [10–12]. In this way, keys and values of tokens not selected at one decoding step are permanently evicted from the KV cache and never recalled in later inference steps, as shown in Fig. 1b.

However, we observe that the token importance changes dynamically during inference. Token that are unimportant with low attention weights at one decoding step can become important in later steps, and vice versa. Figure 3a shows changes in attention weight rankings during 64 decoding steps of Llama-3-8B. For instance, token 3200 is initially unimportant but becomes crucial after 20 steps, while the opposite occurs for token 2048. And importance of all tokens can fluctuate throughout inference, as seen with token 7168. Therefore, non-recallable compression inevitably overlooks some tokens with dynamically changing importance, leading to reduced model accuracy and a decline in output quality.

Defects of existing recallable KV compression methods. However, achieving recallable compression by computing attention weights with all previous tokens incurs an unacceptable cost of $O(Ld)$, which is comparable to attention computation with full KV cache and negates the benefits of compression. To reduce the cost, InfiniGen [18] reduces the dimensions of q and K by generating partial query and key weights offline using singular value decomposition, and projecting hidden states with these partial weights. However, it requires generating

and storing partial keys in addition to original keys, and the selection cost still scales linearly with the context length L.

Quest [15] selects tokens at the granularity of pages, which consist of several consecutive tokens, as depicted in Fig. 1c. To estimate importance of a page, it uses the per-channel maximal keys of all tokens within a page to compute attention weights. This approach reduces the selection cost to $O(Ld/page\_size)$ as only one attention weight is needed for every $page\_size$ tokens. However, recalling pages which are simply divided by token positions leads to internal fragmentation of important tokens. A selected page may contain only a few important tokens, while including unimportant ones for attention computation and wasting the KV cache budget. We illustrate this issue using attention heatmaps of Llama3-8B with an 8k context length in Fig. 3b, where lighter cells represent more important tokens with higher attention weights. As shown, each page of 16 tokens contains only one or two important tokens, and in some cases, two pages or 32 tokens are required to include two consecutive important tokens.

III. Algorithm Design

A. Problem Formulation and Design Rationale

Problem Formulation. For approximated attention computation with selected tokens formulated in Sec. II-B, let K_S to be $(k_{i_1}, k_{i_1}, ..., k_{i_B})^T$, where $I_T = \{i_1, i_2, ..., i_B\}$ denotes the indices of selected tokens. Our goal is to select tokens which contribute most to attention weights, thereby approximating the original computation as closely as possible. Specifically, I_T is supposed to be $\arg\max \sum_{i \in I_T} qk_i^T$, which corresponds to selecting tokens with top-B largest attention weights.

Design Rationale. Our observation is that *tokens which are close in semantic space tend to have similar attention weights for a given q*. Therefore, we propose KV selection at the granularity of semantic clusters. We first apply clustering to tokens in the semantic space. Then, we compute attention weights only with respect to the cluster representations, rather than individual tokens, and select clusters with the largest attention weights. Since the number of clusters is typically an order of magnitude smaller than the number of tokens, cluster-based selection significantly reduces recall overhead.

B. Clustering in the Semantic Space

Semantic Distance. As for a given q, attention weights are associated only with the key tensors, we measure the semantic distance between tokens by calculating the distance between corresponding key vectors. Furthermore, we find that *cosine similarity* is more suitable than L2 or inner product distances. This is due to the presence of outlier channels with large magnitudes in key vectors [19], which can cause drastic changes in L2 or inner product distances. Thus, we define distance between token i and token j in the semantic space as $\mathcal{D}(i,j) = 1 - \frac{\langle k_i, k_j \rangle}{|k_i| \cdot |k_j|}$, where distance is smaller for vectors with larger cosine similarity.

Clustering. We apply a simple K-means algorithm for clustering over key vectors as shown in Fig. 4 [20]. We first randomly sample key vectors as the initial centroids.

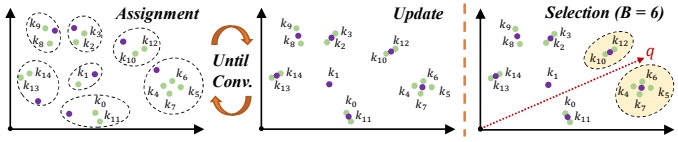

Fig. 4: Clustering and selection process. Green dots represent key vectors, and purple dots represent centroids of clusters.

Subsequently, we alternatively perform the assignment and update steps until convergence. In the assignment step, each key vector is assigned to the nearest centroid and given a corresponding *cluster label* based on the distance $\mathcal{D}$, i.e., assigned to the centroid with the maximum cosine similarity. Then in the update step, the mean of keys assigned to the same centroid are used as the new centroid. The algorithm converges when the assignment no longer changes, and keys assigned to the same centroid form a *semantic cluster*, with the centroid as the cluster representation.

During LLM inference, we first apply clustering to the key vectors of prompt tokens *after the prefill stage*. However, we note an exception for the initial tokens, referred to as attention sinks [9]. These tokens typically appear as outliers in the clustering process, as they are distant from other tokens in the semantic space. Therefore, we always retain the first 16 tokens and apply clustering to the subsequent tokens, generating C_0 centroids. We set $C_0 = \frac{L}{80}$, as our experiments indicate that for a 32k context, using 400 clusters achieves a balance between efficiency and accuracy. For tokens generated *in the decoding stage*, clustering is applied every m decoding steps to the key vectors of m generated tokens, creating C_+ new centroids. Since clustering keys of generated tokens together with those from the prefill stage can incur significant overhead, we instead apply clustering *within* the generated tokens only. To amortize the cost, we set C_+ and m to 4 and 320, respectively.

C. Selection at the Granularity of Semantic Clusters

Selection. We denote cluster centroids as $\mu_1, \mu_2, ..., \mu_C \in \mathbb{R}^d$. To select important tokens for a given query q, we sort those centroids based on their attention weights, i.e., $q\mu_i^T$, in descending order. While keys are clustered using cosine similarity distance, the distance between query vector and centroids is measured with inner product, as it better aligns with attention weight computation. Intuitively, keys assigned to clusters with centroids that have larger attention weights tend to have larger attention weights for the given q. Therefore, we retrieve the sorted centroids and collect KV of tokens from the corresponding clusters, until the top-B most important tokens are selected, as shown in Fig. 4.

D. Efficiency Concerns

Cluster-based selection avoids the internal fragmentation of important tokens and can achieve higher accuracy compared to page-based selection. However, it incurs efficiency concerns.

Concern 1. For clustering, the computational cost is $O(n_i CLd)$ where n_i is the number of iterations until convergence and C is the number of clusters, which is higher than the cost of obtaining page representations, such as per-channel maximal key vectors in Quest [15], which is $O(Ld)$.

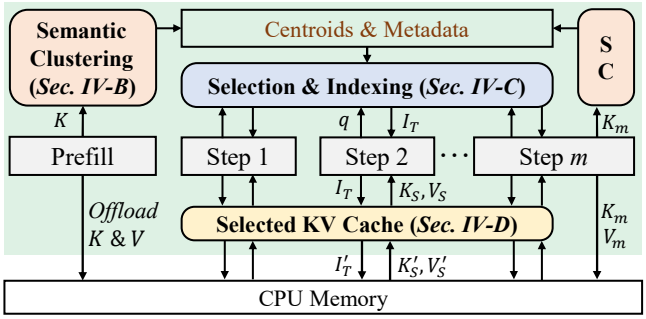

Fig. 5: System overview of ClusterKV. The green box represents components running on the GPU.

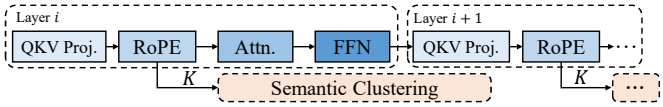

Fig. 6: Overlapping of clustering with other operations.

Concern 2. For selection in page-based methods (Fig. 1c), since page size is fixed and tokens within a page are consecutive, the number of needed pages can be easily computed as $\frac{B}{page\_size}$, and the indices of selected tokens can be directly derived from the indices of selected pages. However, for semantic clusters, the sizes can vary, and the token positions within a cluster are dynamic and discontinuous. Therefore, the number of clusters needed and the indices of selected tokens cannot be easily determined in the same way as for pages.

Our system design addresses both concerns in Sec. IV.

IV. System Design & Implementation

A. System Overview

The system overview is shown in Fig. 5. During *the prefill stage*, the key tensors are processed with semantic clustering (SC) on GPU, producing centroids and corresponding metadata. The generated KV tensors are offloaded to CPU memory. During *the decoding stage*, as described in Section III-C, attention weights of the query vector q and cluster centroids are computed to determine the importance of each cluster. The results, along with clustering metadata, are used to generate indices (I_T) for selected tokens, which are then used to load selected KV (K_S, V_S) from CPU memory to GPU memory for attention computation. A cache for selected KV is maintained on GPU, so only KV not already cached (K'_S, V'_S) need to be loaded. For every m decoding steps, the clustering and KV offloading are performed for the m generated tokens.

B. Semantic Clustering

ClusterKV optimizes the efficiency of clustering at both system and kernel levels.

System-level Optimization. As shown in Fig. 6, ClusterKV applies clustering asynchronously, launching intermediately after the keys are computed from the QKV projection and RoPE modules. This allows clustering to be overlapped with other operations, including attention and FFN computation of the current layer, as well as the QKV projection and RoPE of the following layer. By overlapping these processes, ClusterKV minimizes the overhead associated with clustering.

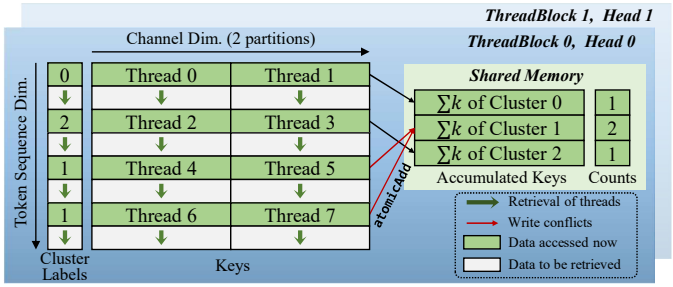

Fig. 7: Execution of the centroid update kernel. In this example, a *ThreadBlock* consists of 8 threads ($BlockSize = 8$), and the channel dimension is divided into two partitions.

Kernel-level Optimization. Since clustering needs to be applied individually for each head, a key optimization is to achieve batched clustering across heads. For the assignment step, which primarily involves `argmin` operations and matrix multiplications between keys and centroids, efficient batched Torch kernels are available [21]. Therefore, we focus on optimizing the centroid update step and implement a custom CUDA kernel where different heads are processed in parallel by individual *ThreadBlocks*, as shown in Fig. 7. The kernel retrieves keys, accumulates keys assigned to the same cluster, records corresponding accumulation counts in the shared memory, and computes the means as new centroids.

A potential issue is write conflict in shared memory, as accumulated keys and counts of the same cluster need to be computed using `atomicAdd`. To mitigate this, we apply several optimizations. As shown in Fig. 7, since distant tokens tend to be assigned to different clusters, we arrange threads in a strided manner along the sequence dimension to minimize the occurrence of concurrent processing of keys within the same cluster. Moreover, we partition the channel dimension into P partitions, with each partition processed by one thread. This results in $\frac{BlockSize}{P}$ keys being processed in parallel. There is a trade-off when choosing P: using a larger P reduces the number of keys processed in parallel and increases the number of iterations along the sequence dimension, while using a smaller P increases the number of iterations along the channel dimension within a thread, and potentially increases the occurrence of write conflicts. To determine the optimal value of P, we perform offline profiling of different P values. We find that for $BlockSize = 512$ using $P = 16$ or $P = 32$ for 128 channels yields optimal performance.

C. Selection and Indexing

The details are shown in Fig. 8. After clustering, ClusterKV stores cluster centroids and corresponding metadata, including cluster sizes, prefix sum, and sorted indices. In this example, k_0 and k_5 are assigned to cluster 2, k_1 is assigned to cluster 0, and k_2, k_3 and k_4 are assigned to cluster 1. The sizes of three clusters can then be obtained. Cluster labels and key indices are sorted by labels, and the sorted indices are stored.

During decoding, ClusterKV computes attention weights between the query vector q and cluster centroids μ, sorting the clusters in descending order of attention weights to determine

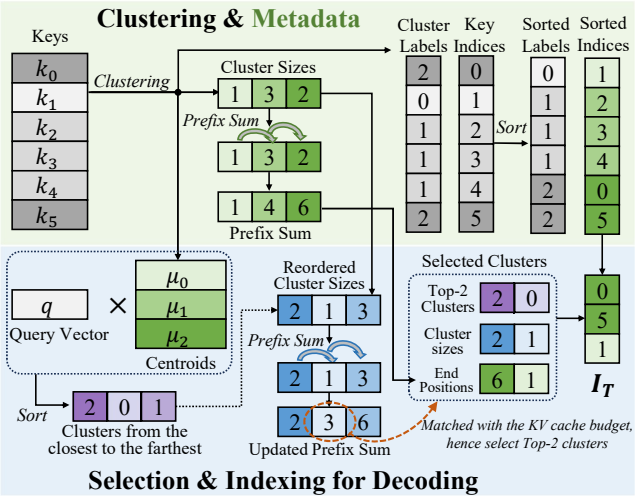

Fig. 8: Process of selection and indexing. Green boxes represent stored metadata from clustering. Here, KV budget is 3.

the closest clusters to q. Next, ClusterKV gathers and reorders their corresponding cluster sizes, and compute prefix sum. If we set the KV cache budget to three, which matches the second prefix sum, top-2 closest clusters will be selected. Then the labels, sizes, and end positions of selected clusters are gathered to determine the indices of selected tokens, I_T. Note that for cases where the summation of selected cluster sizes exceeds the budget, ClusterKV trims tokens from the last selected cluster to adhere to the budget limit.

Similarly to clustering, ClusterKV implements efficient CUDA kernels, processing the indexing of KV heads in parallel with individual *ThreadBlocks*. Metadata related to cluster sizes, which is accessed frequently, is stored in shared memory for improving performance.

D. Caching KV of Selected Tokens

ClusterKV maintains a cluster-granularity cache on GPU to store KV of selected tokens, reducing unnecessary data transfers from CPU memory to GPU memory and enhancing performance. During the decoding stage, the cache retains the KV of selected tokens from the last R decoding steps, along with the labels of corresponding selected clusters. At the current decoding step, the labels of selected clusters are compared with the cluster labels reserved by the cache. Only the KV of clusters not present in the cache are loaded from CPU memory. The cache introduces a trade-off between memory usage and caching effectiveness. In practice, we find that setting $R = 1$, i.e., retaining only the KV states from the last decoding step, strikes a good balance.

V. Evaluation

A. Experimental Setup

We evaluate ClusterKV on an NVIDIA Ada 6000 GPU. To assess model accuracy and output quality, we conduct experiments on eight datasets from LongBench [1], including 2WikiMQA, TriviaQA, HotpotQA, MultiFieldQA, MuSiQue, NarrativeQA, Qasper and GovReport. These datasets cover a variety of tasks, such as single-doc QA, multi-doc QA,

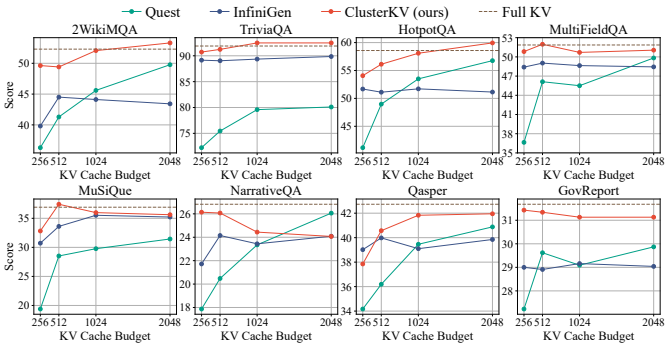

Fig. 9: Results on LongBench of different methods.

TABLE I: Average scores on eight LongBench datasets.

Method \ Budget	256	512	1024	2048
Quest	35.63	40.83	43.23	45.59
InfiniGen	43.69	45.04	45.13	45.14
ClusterKV (ours)	**46.69**	**48.02**	**48.34**	**48.7**
Full KV	49.01			

few shot learning and summarization, with context lengths reaching up to 32k. Additionally, we evaluate ClusterKV on the PG19 dataset for the language modeling task [22]. We use the state-of-the-art long-context GLM4-9B-Chat model for evaluation of model accuracy [7], which supports a context window of up to 128k tokens.

We compare ClusterKV with two state-of-the-art KV cache compression methods: Quest [15] and InfiniGen [18]. Most configurations of these methods remain as in their original settings, such as *page_size* for Quest and the partial weight ratio and selection threshold for InfiniGen. To align with settings of Quest which does not apply selection on the first two layers of the model, we also disable selection and use the full KV cache for the first two layers in ClusterKV and InfiniGen, in both model accuracy and inference performance evaluations. As both Quest and InfiniGen only support efficient inference for Llama-architecture models, we use Llama-3.1-8B for evaluation of inference performance.

B. Model Accuracy

Results on LongBench. Figure 9 presents the results on LongBench. ROUGE-L is used as the score metric for GovReport and F1 score is used for other tasks. We evaluate methods under the KV cache budgets of 256, 512, 1024 and 2048 tokens. ClusterKV outperforms Quest and InfiniGen in most settings, and achieves accuracy comparable to that of using the full KV cache with budgets as low as 1k to 2k tokens.

The average scores across the eight tasks under different budgets are summarized in Table I, where ClusterKV demonstrates significant improvement over Quest and InfiniGen.

Results on Language Modeling. We report the perplexity of the language modeling task in Fig. 10. The prompts are from the PG19 test set, with input lengths ranging from 1 to 32000 tokens. The KV cache budget is uniformly set to 1024 for ClusterKV, Quest, and InfiniGen. As shown, ClusterKV closely aligns with the full KV, with a perplexity deviation

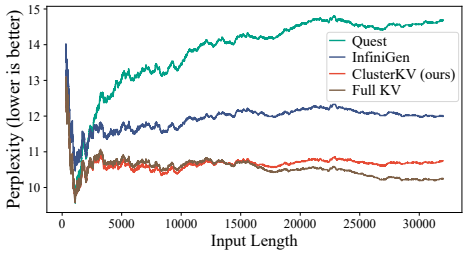

Fig. 10: Perplexity of language modeling with a KV cache budget of 1024 tokens.

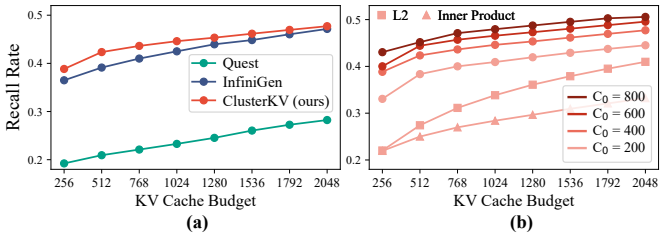

Fig. 11: Recall rate of important tokens: (a) comparison between different methods, and (b) comparison between different configurations of ClusterKV.

up to 0.5, while Quest and InfiniGen show deviations of approximately 4 and 2, respectively.

Recall Rate of important tokens. We extract a sample from the NarrativeQA dataset with a context length of 32k and compute the recall rates of important tokens during inference for different methods. The recall rate is defined as $\frac{|I_T \cap I_T^{true}|}{|I_T^{true}|}$, where $|I_T| = |I_T^{true}| = B$, I_T represents the indices of selected tokens, and I_T^{true} denotes the indices of tokens with the top-B largest attention weights. We report the recall rates averaged across layers, heads and decoding steps, and the budget B is varied from 256 to 2048 in increments of 256.

As shown in Fig. 11a, ClusterKV achieves higher recall rates across all budgets. We further explore the impact of different configurations for ClusterKV, including clustering distance metrics and the number of clusters C_0. In Fig. 11b, cosine similarity used by ClusterKV outperforms both L2 and inner product distance. In addition, increasing C_0 improves recall rates, while the improvements become less significant when $C_0 > 400$. Consequently, we use $C_0 = 400 = \frac{L}{80}$ as a balanced choice for accuracy and efficiency.

C. Inference Efficiency

Comparison with inference using full KV cache. We evaluate the inference latency of ClusterKV under various KV cache budgets and compare it against the full KV cache configuration. The experiments are conducted with prompt lengths (P) ranging from 8k to 32k and decoding lengths (D) ranging from 256 to 1024. As shown in Fig. 12, ClusterKV achieves significant efficiency improvements. For $P = 32k$ and $D = 1024$, the speedup in latency is 2× with a budget of 1024 tokens, respectively. Additionally, the decoding throughput increases by up to 2.5×. We also analyze the time spent during prefill. As shown, the clustering overhead of ClusterKV

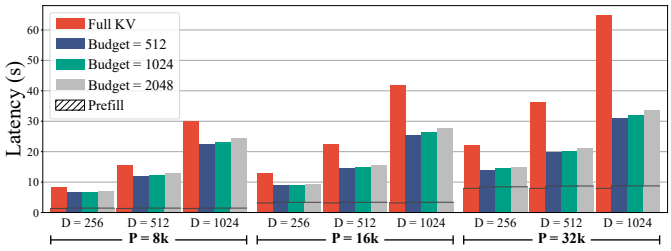

Fig. 12: Comparison of inference latency between ClusterKV and the full KV cache configuration.

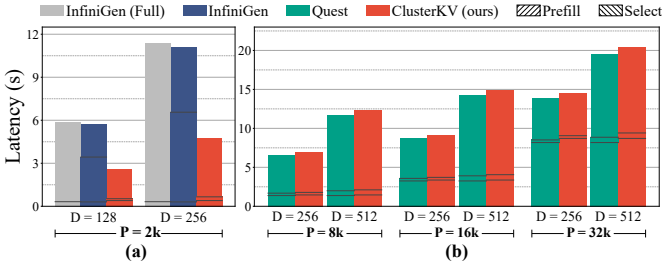

Fig. 13: Comparison of inference latency: (a) ClusterKV vs. InfiniGen with a budget of 256 tokens, and (b) ClusterKV vs. Quest with a budget of 1k tokens.

is minimal, accounting for only 6% to 8% of prefill and less than 2% of the total inference time.

Comparison with SoTA compression methods. InfiniGen is implemented based on FlexGen [23], which exclusively supports OPT [24] models with a 2k context window from scratch and is hard to adapt to other models, so we compare ClusterKV with InfiniGen using OPT-6.7B, as shown in Fig. 13a. ClusterKV achieves an average speedup of 2.3×. InfiniGen's latency is comparable to that of inference using the full KV cache, due to the high computational cost of its per-token selection, while ClusterKV's selection overhead accounts for only about 5% of the total decoding latency.

We present the comparison between ClusterKV and Quest in Fig. 13b, using Llama-3.1-8B model with a budget of 1k tokens. As shown, ClusterKV achieves performance very close to Quest, with latency deviations up to 5%, while ClusterKV delivers significantly higher model accuracy.

Effectiveness of caching. We analyze the hit rates of our cluster-granularity cache during inference, using a 32k-tokens sample from the NarrativeQA dataset. The average hit rates are 63% and 74% for $R = 1$ and $R = 2$, respectively. Compared to directly loading from CPU memory, the caching mechanism improves decoding throughput by 2.3× and 3×, respectively.

VI. Conclusion

This paper introduces ClusterKV, achieving efficient and accurate KV cache compression, recalling tokens at the granularity of semantic clusters. ClusterKV preserves model accuracy while significantly enhancing inference efficiency.

VII. Acknowledgment

This work is sponsored by the National Natural Science Foundation of China (62472273, 62232015).

REFERENCES

[1] Y. Bai, X. Lv, J. Zhang, H. Lyu, J. Tang, Z. Huang, Z. Du, X. Liu, A. Zeng, L. Hou, Y. Dong, J. Tang, and J. Li, "Longbench: A bilingual, multitask benchmark for long context understanding," *arXiv preprint arXiv:2308.14508*, 2023.

[2] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. R. Narasimhan, "SWE-bench: Can language models resolve real-world github issues?" in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=VTF8yNQM66

[3] OpenAI, "Openai o1 system card," 2024. [Online]. Available: https://cdn.openai.com/o1-system-card-20240917.pdf

[4] Meta, "Llama 2: Open foundation and fine-tuned chat models," 2023. [Online]. Available: https://arxiv.org/abs/2307.09288

[5] D. Li, R. Shao, A. Xie, Y. Sheng, L. Zheng, E. Gonzalez, I. Stoica, X. Ma, and H. Zhang, "How long can open-source llms truly promise on context length?" June 2023. [Online]. Available: https://lmsys.org/blog/2023-06-29-longchat

[6] OpenAI, "Gpt-4o system card," 2024. [Online]. Available: https://cdn.openai.com/gpt-4o-system-card.pdf

[7] T. GLM, A. Zeng, B. Xu, B. Wang, C. Zhang, D. Yin, D. Rojas, G. Feng, H. Zhao, H. Lai, H. Yu, H. Wang, J. Sun, J. Zhang, J. Cheng, J. Gui, J. Tang, J. Zhang, J. Li, L. Zhao, L. Wu, L. Zhong, M. Liu, M. Huang, P. Zhang, Q. Zheng, R. Lu, S. Duan, S. Zhang, S. Cao, S. Yang, W. L. Tam, W. Zhao, X. Liu, X. Xia, X. Zhang, X. Gu, X. Lv, X. Liu, X. Liu, X. Yang, X. Song, X. Zhang, Y. An, Y. Xu, Y. Niu, Y. Yang, Y. Li, Y. Bai, Y. Dong, Z. Qi, Z. Wang, Z. Yang, Z. Du, Z. Hou, and Z. Wang, "Chatglm: A family of large language models from glm-130b to glm-4 all tools," 2024.

[8] P. Patel, E. Choukse, C. Zhang, A. Shah, I. Goiri, S. Maleki, and R. Bianchini, "Splitwise: Efficient generative llm inference using phase splitting," in *ISCA*, June 2024.

[9] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient streaming language models with attention sinks," *ICLR*, 2024.

[10] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett, Z. A. Wang, and B. Chen, "H2o: Heavy-hitter oracle for efficient generative inference of large language models," in *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., vol. 36. Curran Associates, Inc., 2023, pp. 34 661–34 710.

[11] Y. Li, Y. Huang, B. Yang, B. Venkitesh, A. Locatelli, H. Ye, T. Cai, P. Lewis, and D. Chen, "Snapkv: Llm knows what you are looking for before generation," 2024. [Online]. Available: https://arxiv.org/abs/2404.14469

[12] M. Adnan, A. Arunkumar, G. Jain, P. J. Nair, I. Soloveychik, and P. Kamath, "Keyformer: Kv cache reduction through key tokens selection for efficient generative inference," *MLSys*, 2024.

[13] Y. Zhao, D. Wu, and J. Wang, "Alisa: Accelerating large language model inference via sparsity-aware kv caching," 2024. [Online]. Available: https://arxiv.org/abs/2403.17312

[14] Z. Ning, J. Zhao, Q. Jin, W. Ding, and M. Guo, "Inf-mllm: Efficient streaming inference of multimodal large language models on a single gpu," 2024. [Online]. Available: https://arxiv.org/abs/2409.09086

[15] J. Tang, Y. Zhao, K. Zhu, G. Xiao, B. Kasikci, and S. Han, "Quest: Query-aware sparsity for efficient long-context llm inference," *ICML*, 2024.

[16] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," 2023. [Online]. Available: https://arxiv.org/abs/1706.03762

[17] K. Ataallah, C. Gou, E. Abdelrahman, K. Pahwa, J. Ding, and M. Elhoseiny, "Infinibench: A comprehensive benchmark for large multimodal models in very long video understanding," 2024. [Online]. Available: https://arxiv.org/abs/2406.19875

[18] W. Lee, J. Lee, J. Seo, and J. Sim, "InfiniGen: Efficient generative inference of large language models with dynamic KV cache management," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. Santa Clara, CA: USENIX Association, Jul. 2024, pp. 155–172. [Online]. Available: https://www.usenix.org/conference/osdi24/presentation/lee

[19] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, and X. Hu, "Kivi: A tuning-free asymmetric 2bit quantization for kv cache," *ICML*, 2024.

[20] A. M. Ikotun, A. E. Ezugwu, L. Abualigah, B. Abuhaija, and J. Heming, "K-means clustering algorithms: A comprehensive review, variants analysis, and advances in the era of big data," *Information Sciences*, vol. 622, pp. 178–210, 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0020025522014633

[21] J. Ansel, E. Yang, H. He, N. Gimelshein, A. Jain, M. Voznesensky, B. Bao, P. Bell, D. Berard, E. Burovski, G. Chauhan, A. Chourdia, W. Constable, A. Desmaison, Z. DeVito, E. Ellison, W. Feng, J. Gong, M. Gschwind, B. Hirsh, S. Huang, K. Kalambarkar, L. Kirsch, M. Lazos, M. Lezcano, Y. Liang, J. Liang, Y. Lu, C. K. Luk, B. Maher, Y. Pan, C. Puhrsch, M. Reso, M. Saroufim, M. Y. Siraichi, H. Suk, S. Zhang, M. Suo, P. Tillet, X. Zhao, E. Wang, K. Zhou, R. Zou, X. Wang, A. Mathews, W. Wen, G. Chanan, P. Wu, and S. Chintala, "Pytorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS '24. New York, NY, USA: Association for Computing Machinery, 2024, pp. 929–947. [Online]. Available: https://doi.org/10.1145/3620665.3640366

[22] J. W. Rae, A. Potapenko, S. M. Jayakumar, and T. P. Lillicrap, "Compressive transformers for long-range sequence modelling," 2019. [Online]. Available: https://arxiv.org/abs/1911.05507

[23] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, D. Y. Fu, Z. Xie, B. Chen, C. Barrett, J. E. Gonzalez, P. Liang, C. Ré, I. Stoica, and C. Zhang, "Flexgen: High-throughput generative inference of large language models with a single gpu," 2023. [Online]. Available: https://arxiv.org/abs/2303.06865

[24] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. V. Lin, T. Mihaylov, M. Ott, S. Shleifer, K. Shuster, D. Simig, P. S. Koura, A. Sridhar, T. Wang, and L. Zettlemoyer, "Opt: Open pre-trained transformer language models," 2022. [Online]. Available: https://arxiv.org/abs/2205.01068